

2026 春《操作系统》课程实验报告

实验 0

23301037 客昱阳

一.环境准备

1.git 环境准备

- 初始化 Git 仓库

Bash

```
1 git init
```

- 创建 `.gitignore`

Bash

```
1 touch .gitignore
2 echo .DS_Store >> .gitignore
```

- 关联远程仓库

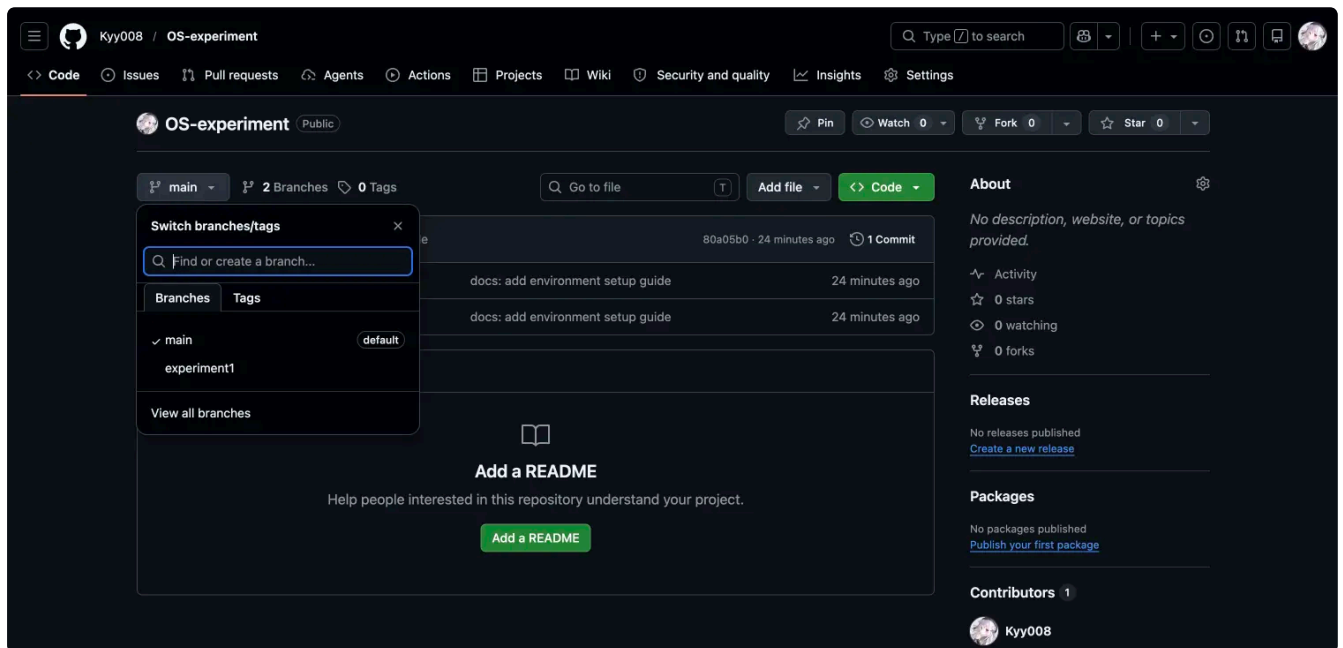
Bash

```
1 git remote add origin https://github.com/Kyy008/OS-experiment
```

- 创建并推送实验一分支

Bash

```
1 git branch exp1
2 git push -u origin exp1
```



- 切换到实验一分支

Bash

```
1 git checkout experiment1
```

2.Docker 环境准备

- 拉取 openEuler 镜像

Bash

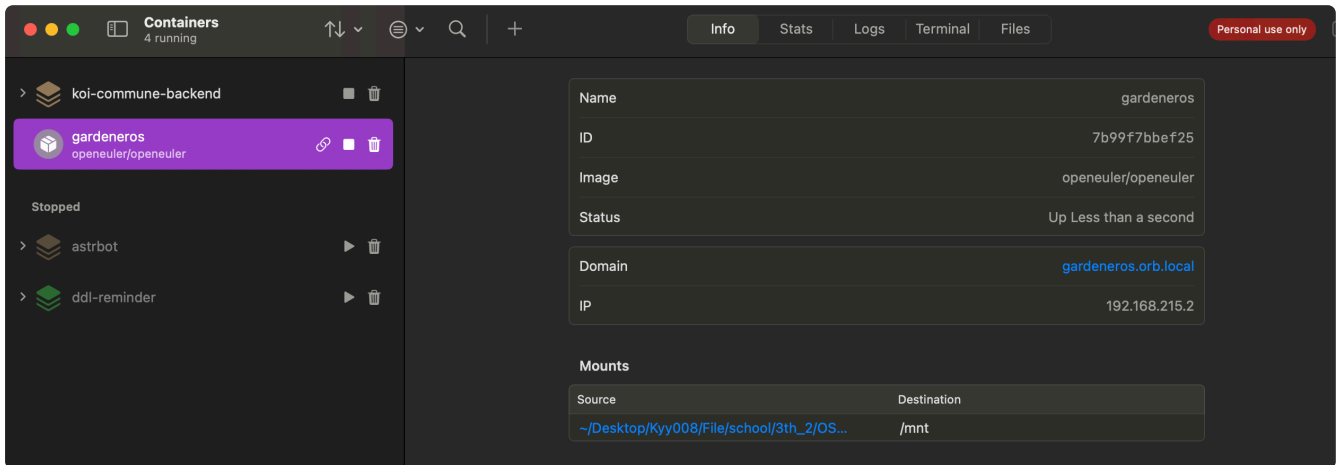
```
1 docker pull openeuler/openeuler
```

```
~/De/K/F/school/3th_2/OS/experiment main 0.015s 11:41:05
docker pull openeuler/openeuler
Using default tag: latest
latest: Pulling from openeuler/openeuler
c2a5d1e5887b: Pull complete
ba0e6af23b31: Pull complete
Digest: sha256:990f5a8528dc3375a358629bcb5500351f433a49922dca3588bcf32ecc5b7022
Status: Downloaded newer image for openeuler/openeuler:latest
docker.io/openeuler/openeuler:latest
```

- 在实验目录下启动容器, 把当前的实验目录挂载到容器里的 /mnt

Bash

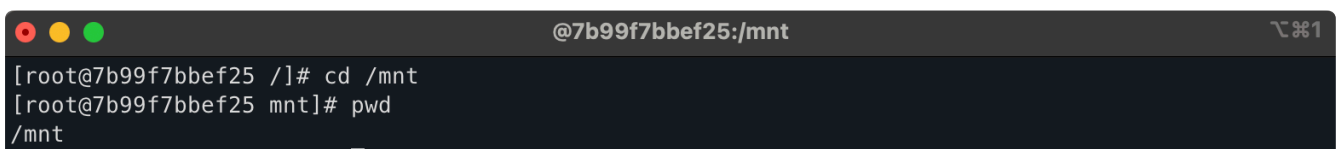
```
1 docker run -it --name gardeneros \  
2   --mount type=bind,source="$(pwd)",destination=/mnt \  
3   openeuler/openeuler /bin/bash
```



- 进入容器内的挂载目录

Bash

```
1 cd /mnt  
2 pwd
```



- 为容器安装基础工具：

Bash

```
1 dnf install -y curl vim gcc git
```



```
systemtap-sdt-devel-5.0-3.oe2403sp2.aarch64
util-linux-2.39.1-35.oe2403sp2.aarch64
vim-common-2:9.0.2092-29.oe2403sp2.aarch64
vim-enhanced-2:9.0.2092-29.oe2403sp2.aarch64
vim-filesystem-2:9.0.2092-29.oe2403sp2.noarch
xkeyboard-config-2.39-4.oe2403sp2.noarch
zip-3.0-32.oe2403sp2.aarch64
```

Complete!

[root@7b99f7bbef25 mnt]#

3.配置 Rust 开发环境

- 使用官方脚本安装 Rust

Bash

```
1 curl https://sh.rustup.rs -sSf | sh
```

Rust is installed now. Great!

To get started you may need to restart your current shell.
This would reload your **PATH** environment variable to include
Cargo's bin directory (\$HOME/.cargo/bin).

To configure your current shell, you need to source
the corresponding **env** file under \$HOME/.cargo.

This is usually done by running one of the following (note the leading DOT):

```
. "$HOME/.cargo/env"      # For sh/bash/zsh/ash/dash/pdksh
source "$HOME/.cargo/env.fish" # For fish
source ~/.cargo/env.nu     # For nushell
source "$HOME/.cargo/env.tcsh" # For tcsh
. "$HOME/.cargo/env.ps1"    # For pwsh
source "$HOME/.cargo/env.xsh" # For xonsh
```

- 执行如下命令，使得安装环境启用，并检查版本

Bash

```
1 source "$HOME/.cargo/env"
2 rustc --version
3 cargo --version
4 rustup --version
```

```
[root@7b99f7bbef25 ~]# source "$HOME/.cargo/env"
rustc --version
cargo --version
rustup --version
rustc 1.95.0 (59807616e 2026-04-14)
cargo 1.95.0 (f2d3ce0bd 2026-03-21)
rustup 1.29.0 (28d1352db 2026-03-05)
info: This is the version for the rustup toolchain manager, not the rustc compiler.
info: the currently active `rustc` version is `rustc 1.95.0 (59807616e 2026-04-14)`
[root@7b99f7bbef25 ~]#
```

- 安装 nightly Rust 工具链

Bash

```
1 rustup install nightly
```

```
[root@7b99f7bbef25 ~]# rustup install nightly
info: syncing channel updates for nightly-aarch64-unknown-linux-gnu
info: latest update on 2026-04-27 for version 1.97.0-nightly (ca9a134e0 2026-04-26)
info: downloading 6 components
   cargo installed             10.04 MiB
   clippy installed            3.62 MiB
  rust-docs installed          22.51 MiB
  rust-std installed           28.83 MiB
   rustc installed             59.55 MiB
  rustfmt installed            1.87 MiB
nightly-aarch64-unknown-linux-gnu installed - rustc 1.97.0-nightly (ca9a134e0 2026-04-26)
info: checking for self-update (current version: 1.29.0)
```

- 把默认 Rust 版本切换成 nightly。

Bash

```
1 rustup default nightly
```

```
@7b99f7bbef25:~
[root@7b99f7bbef25 ~]# rustup default nightly
info: using existing install for nightly-aarch64-unknown-linux-gnu
info: default toolchain set to nightly-aarch64-unknown-linux-gnu

nightly-aarch64-unknown-linux-gnu unchanged - rustc 1.97.0-nightly (ca9a134e0 2026-04-26)
[root@7b99f7bbef25 ~]#
```

- 添加 RISC-V 编译目标

Bash

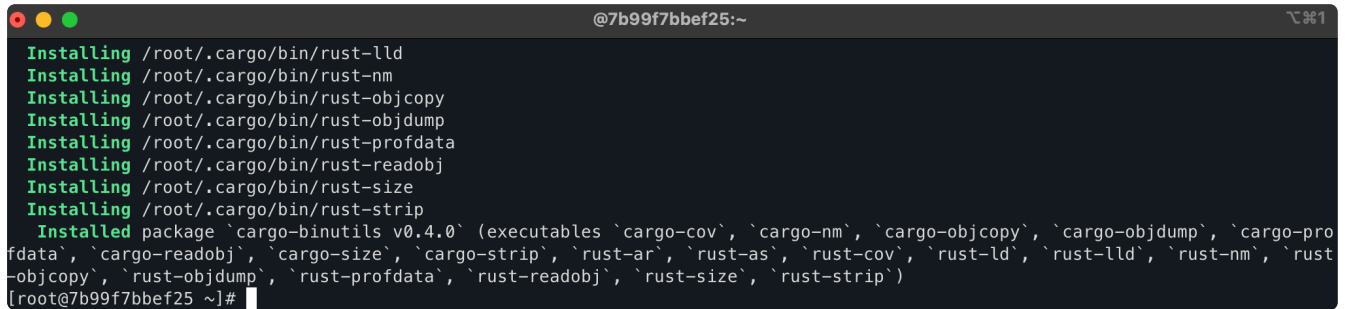
```
1 rustup target add riscv64gc-unknown-none-elf
```

```
[root@7b99f7bbef25 ~]# rustup target add riscv64gc-unknown-none-elf
info: downloading component rust-std
   rust-std installed             11.50 MiB
[root@7b99f7bbef25 ~]#
```

- 安装 `cargo-binutils`

Bash

```
1 cargo install cargo-binutils
```

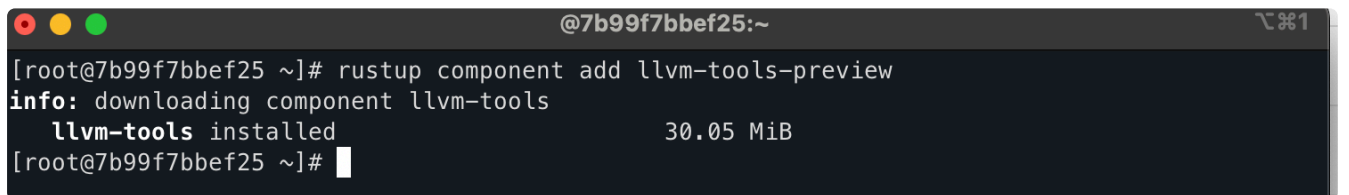


```
@7b99f7bbef25:~  
Installing /root/.cargo/bin/rust-lld  
Installing /root/.cargo/bin/rust-nm  
Installing /root/.cargo/bin/rust-objcopy  
Installing /root/.cargo/bin/rust-objdump  
Installing /root/.cargo/bin/rust-profdata  
Installing /root/.cargo/bin/rust-readobj  
Installing /root/.cargo/bin/rust-size  
Installing /root/.cargo/bin/rust-strip  
Installed package `cargo-binutils v0.4.0` (executables `cargo-cov`, `cargo-nm`, `cargo-objcopy`, `cargo-objdump`, `cargo-profdata`, `cargo-readobj`, `cargo-size`, `cargo-strip`, `rust-ar`, `rust-as`, `rust-cov`, `rust-ld`, `rust-lld`, `rust-nm`, `rust-objcopy`, `rust-objdump`, `rust-profdata`, `rust-readobj`, `rust-size`, `rust-strip`)  
[root@7b99f7bbef25 ~]#
```

- 安装 `llvm-tools-preview` 组件

Bash

```
1 rustup component add llvm-tools-preview
```

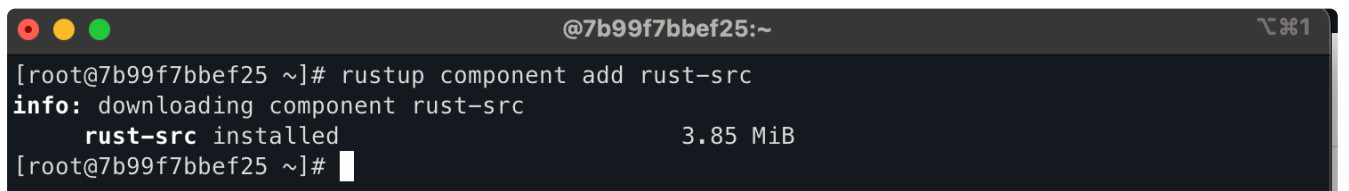


```
@7b99f7bbef25:~  
[root@7b99f7bbef25 ~]# rustup component add llvm-tools-preview  
info: downloading component llvm-tools  
      llvm-tools installed           30.05 MiB  
[root@7b99f7bbef25 ~]#
```

- 安装 `rust-src` 组件

Bash

```
1 rustup component add rust-src
```



```
@7b99f7bbef25:~  
[root@7b99f7bbef25 ~]# rustup component add rust-src  
info: downloading component rust-src  
      rust-src installed            3.85 MiB  
[root@7b99f7bbef25 ~]#
```

- 在工作目录下创建一个名为 `rust-toolchain` 的文件，以限制 rust 的版本

Bash

```
1 cd /mnt  
2 echo nightly-2022-10-19 > rust-toolchain
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 ~]# cd /mnt
[root@7b99f7bbef25 mnt]# echo nightly-2022-10-19 > rust-toolchain
[root@7b99f7bbef25 mnt]# cat rust-toolchain
nightly-2022-10-19
```

4. 安装 qemu

- 安装编译 QEMU 需要的基础开发工具

```
Bash

1 cd ~
2 dnf groupinstall -y "Development Tools"
```

```
@7b99f7bbef25:~
wayland-1.22.0-1.oe2403sp2.aarch64
xorg-x11-font-utils-1:7.5-45.oe2403sp2.aarch64
xorg-x11-fonts-7.5-27.oe2403sp2.noarch
xorg-x11-utils-7.5-31.oe2403sp2.aarch64
xz-5.4.7-8.oe2403sp2.aarch64

Complete!
```

- 安装 QEMU 编译所需的其他依赖

```
Bash

1 dnf install -y autoconf automake gcc gcc-c++ kernel-devel curl libmpc-d
  evel mpfr-devel gmp-devel glib2 glib2-devel make cmake gawk bison flex
  texinfo gperf libtool patchutils bc python3 ninja-build wget xz
```

```
@7b99f7bbef25:~
perl-libintl-perl-1.33-1.oe2403sp2.aarch64
texinfo-7.0.3-4.oe2403sp2.aarch64
util-linux-devel-2.39.1-35.oe2403sp2.aarch64
wget-1.21.4-3.oe2403sp2.aarch64
zlib-devel-1.2.13-5.oe2403sp2.aarch64

Complete!
[root@7b99f7bbef25 ~]#
```

- 下载 QEMU 5.2 源码

```
Bash

1 wget https://download.qemu.org/qemu-5.2.0.tar.xz
```

```
[root@7b99f7bbef25 ~]# wget https://download.qemu.org/qemu-5.2.0.tar.xz
--2026-04-27 09:37:13-- https://download.qemu.org/qemu-5.2.0.tar.xz
Resolving download.qemu.org (download.qemu.org)... 198.18.16.244
Connecting to download.qemu.org (download.qemu.org)|198.18.16.244|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 106902800 (102M) [application/x-tar]
Saving to: 'qemu-5.2.0.tar.xz'

qemu-5.2.0.tar.xz      100%[=====] 101.95M  11.9MB/s   in 9.5s

2026-04-27 09:37:24 (10.8 MB/s) - 'qemu-5.2.0.tar.xz' saved [106902800/106902800]

[root@7b99f7bbef25 ~]#
```

- 解压 QEMU 源码包并进入

Bash

```
1 tar xvf qemu-5.2.0.tar.xz
2 cd qemu-5.2.0
```

```
@7b99f7bbef25:~/qemu-5.2.0
qemu-5.2.0/pc-bios/opensbi-riscv32-generic-fw_dynamic.bin
qemu-5.2.0/pc-bios/edk2-arm-code.fd.bz2
qemu-5.2.0/pc-bios/linuxboot.bin
qemu-5.2.0/configure
qemu-5.2.0/qemu.nsi
qemu-5.2.0/Kconfig
qemu-5.2.0/.gdbinit
[root@7b99f7bbef25 qemu-5.2.0]#
```

- 配置 QEMU 只编译 RISC-V 相关目标并编译安装

Bash

```
1 ./configure --target-list=riscv64-softmmu,riscv64-linux-user && make -j
$(nproc) install
```

```
@7b99f7bbef25:~/qemu-5.2.0
e/qemu/firmware
Installing /root/qemu-5.2.0/build/pc-bios/descriptors/60-edk2-aarch64.json to /usr/local/share/qemu/
firmware
Installing /root/qemu-5.2.0/build/pc-bios/descriptors/60-edk2-arm.json to /usr/local/share/qemu/fir
mware
Installing /root/qemu-5.2.0/build/pc-bios/descriptors/60-edk2-i386.json to /usr/local/share/qemu/fi
rmware
Installing /root/qemu-5.2.0/build/pc-bios/descriptors/60-edk2-x86_64.json to /usr/local/share/qemu/
firmware
Installing /root/qemu-5.2.0/pc-bios/keymaps/sl to /usr/local/share/qemu/keymaps
Installing /root/qemu-5.2.0/pc-bios/keymaps/sv to /usr/local/share/qemu/keymaps
make[1]: Leaving directory '/root/qemu-5.2.0/build'
[root@7b99f7bbef25 qemu-5.2.0]#
```

- 验证 QEMU 安装成功


```
Bash
1 qemu-system-riscv64 --version
2 qemu-riscv64 --version
```

```
Bash
1 qemu-system-riscv64 --version
2 qemu-riscv64 --version
```

```
@7b99f7bbef25:~/qemu-5.2.0
[root@7b99f7bbef25 qemu-5.2.0]# qemu-system-riscv64 --version
qemu-riscv64 --version
QEMU emulator version 5.2.0
Copyright (c) 2003-2020 Fabrice Bellard and the QEMU Project developers
qemu-riscv64 version 5.2.0
Copyright (c) 2003-2020 Fabrice Bellard and the QEMU Project developers
[root@7b99f7bbef25 qemu-5.2.0]#
```

- 把配置的环境变成自己的 docker 镜像

```
Bash
1 docker commit -m "configure os experiment environment" -a "Kyy008" gard
  eneros myopeneuler
```

```
Bash
1 docker commit -m "configure os experiment environment" -a "Kyy008" gard
  eneros myopeneuler
```

```

~ /De/K/F/school/3th_2/OS/experiment main ? 0.009s 18:09:20
docker commit -m "configure os experiment" -a "Kyy008" gardeneros myopeneuler
sha256:cdf7823a9112513378b4f4941d4f6f80093055aea61e68e74d877d95032b8915

```

[illegible]